

Security Audit

PoolParty (dApp)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	41
Conclusion	42
Our Methodology	43
Disclaimers	45
About Hashlock	46

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The PoolParty team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

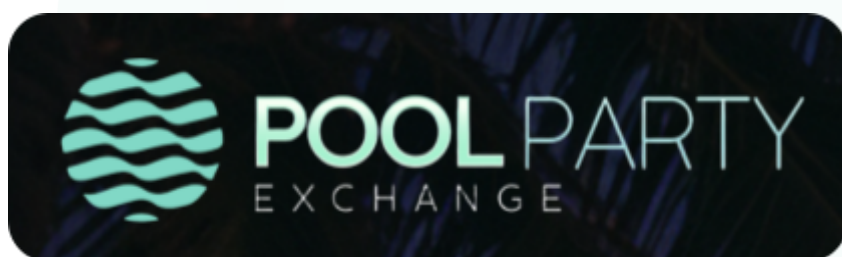
PoolParty™ is a decentralized trading platform combining DEX and OTC features. It allows users to list and trade tokens peer-to-peer with no price impact or fees. Buyers can access discounted tokens, while sellers earn ETH rewards. Users can also stake \$SPLASH to earn passive income.

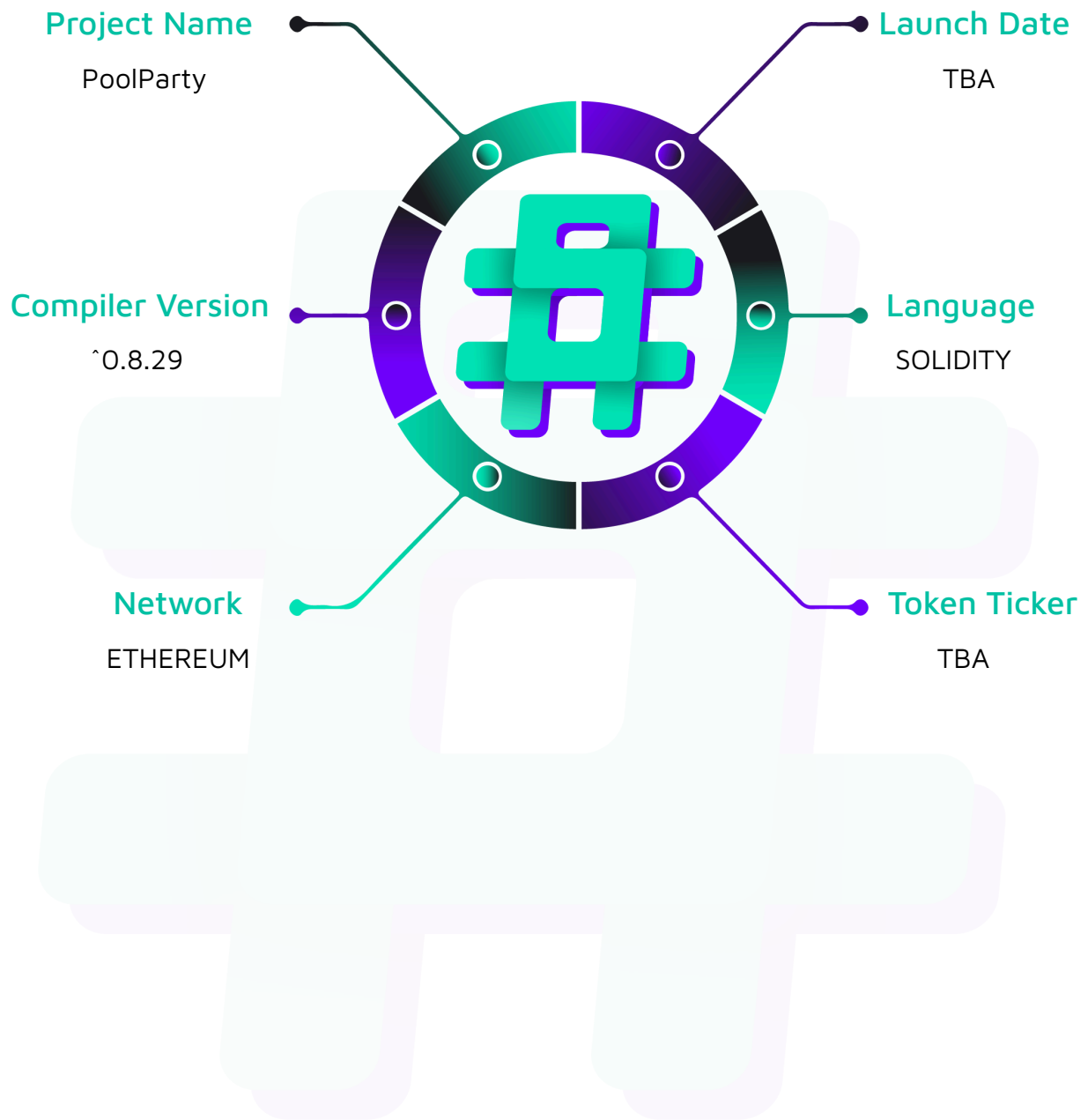
Project Name: PoolParty

Project Type: dApp

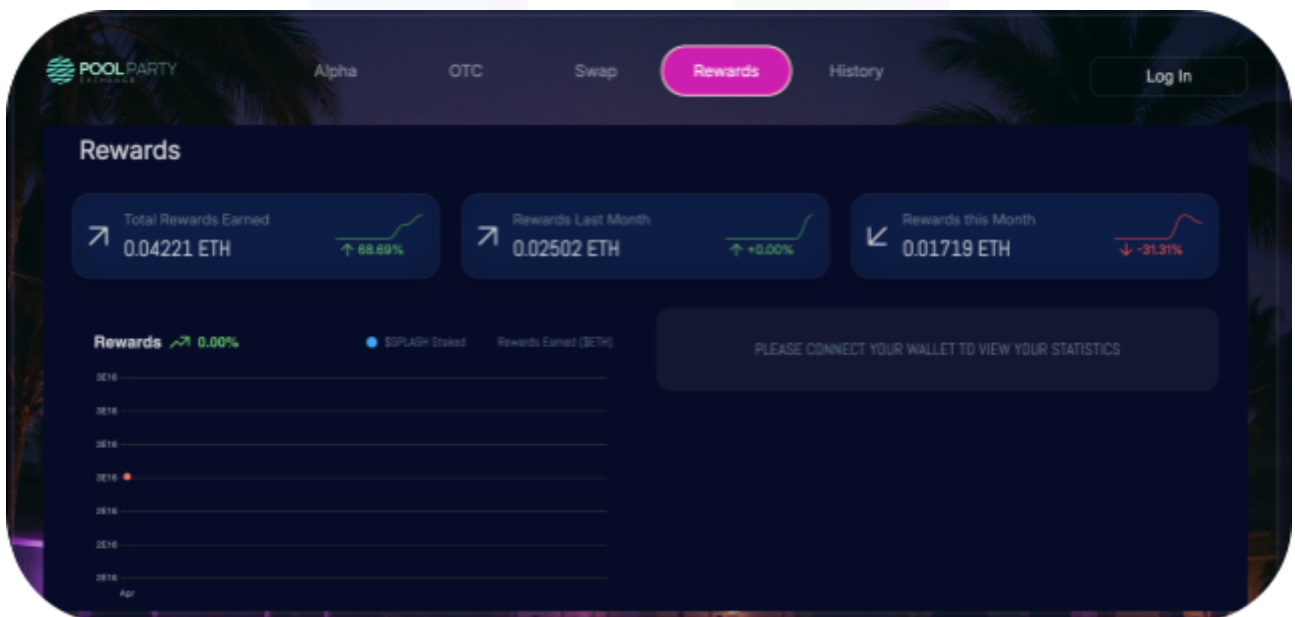
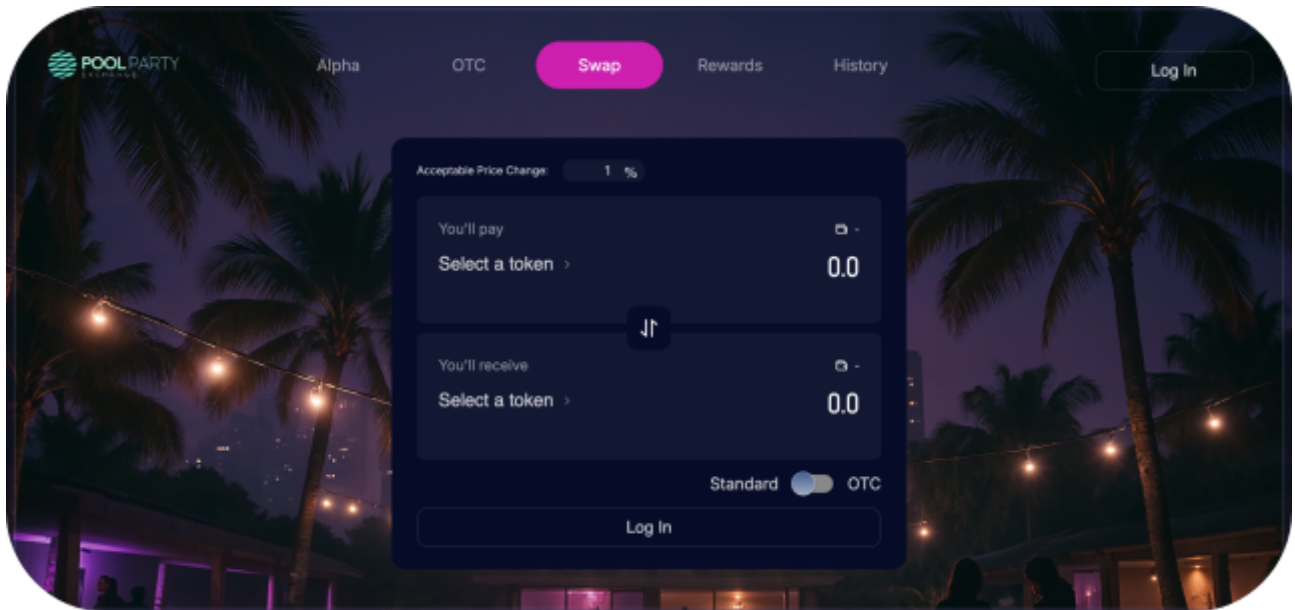
Compiler Version: ^0.8.29

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the PoolParty project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	PoolParty Smart Contracts
Platform	EVM / Solidity
Audit Date	June, 2025
Contract 1	PoolPartySwap.sol
Contract 2	PoolPartyStaking.sol
Contract 3	publicPool.sol
Contract 4	ownedPool.sol
Contract 5	publicPoolContract.sol
Contract 6	ownedPoolContract.sol
Contract 7	resellerCommunityRewards.sol
Contract 8	resellerCommunityRewardsL2.sol
Contract 9	PoolPartyStructs.sol
Contract 10	UpdatePoolPartyStruct.sol
Contract 11	StargateAddresses.sol
Contract 12	RouterAddresses.sol
Contract 13	SupportedAssetsManager.sol
Contract 14	PriceOracleAddresses.sol
Contract 15	PoolPartyEvents.sol
Contract 16	BokkyPooBahsDateTimeLibrary.sol
Contract 17	IPoolParty.sol

Contract 18	IPoolPartyStaking.sol
Contract 19	IResellerRewards.sol
Contract 20	IPoolPartyContract.sol
Contract 21	IPublicPoolContract.sol
Contract 22	IOwnedPoolContract.sol
Contract 23	IOracle.sol
Contract 24	IPartyX.sol
Contract 25	IPoolPartyEvents.sol
Audited GitHub Commit Hash	8ea8965c9f986f5ce06a29678fe4d39599640276
Fix Review GitHub Commit Hash	e9da8f2e06ffec03d5210943c6e8d608c95b05e3

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contract follows simple logic, with correct and detailed ordering. The protocol uses a list of Open Zeppelin contracts.

**Not Secure****Vulnerable****Secure****Hashlocked**

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities we have identified have been resolved or acknowledged.

Hashlock found:

2 High severity vulnerabilities

6 Medium severity vulnerabilities

3 Low severity vulnerabilities

10 Gas Optimisations

4 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
PoolPartySwap.sol - Core contract managing pools, feels and platform functionalities	Contract achieves this functionality.
PoolPartyStaking.sol - Handles SLPASH tokens minting and staking rewards - Handles SLPASH and PartyX tokens conversions	Contract achieves this functionality under certain conditions
ownedPool.sol - Implement single-owner token pools with optional vesting	Contract achieves this functionality.
publicPool.sol - Implement many-to-many token pools with queue-base distributions	Contract achieves this functionality.
ownedPoolContract.sol - Factory for creating ownedPool contracts	Contract achieves this functionality.
publicPoolContract.sol - Factory for creating publicPool contracts	Contract achieves this functionality.
resellerCommunityRewardsL2.sol - Bridges community rewards from L2 to mainnet after receiving the rewards in ETH and swapping them to stablecoins	Contract achieves this functionality.
resellerCommunityRewards.sol - Distribute ETH rewards to SPLASH stakers on Ethereum	Contract achieves this functionality under certain conditions.

Code Quality

This audit scope involves the smart contracts of the PoolParty project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the PoolParty project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] PoolPartyStaking#sacrifice - Some users might end up getting more rewards at the expense of others because of the incorrect updates of `stakedPartyXBalance` and `totalPartyXStaked`

Description

As stated in the natspec of the `sacrifice(..)` function, the sole source of documentation for this functionality:

- * @dev Sacrifices PartyX tokens to receive SPLASH tokens at a special rate*
- * This function transfers PartyX tokens from the user to this contract,*
- * burns them, and mints SPLASH tokens at the sacrifice rate (typically greater than 1:1).*

This means that users can burn their PartyX tokens to get SPLASH tokens at a more interesting rate, this should not involve staking.

Vulnerability Details

`stakedPartyXBalance` and `totalPartyXStaked` should be left untouched in this function. These 2 state variables are increasing in value but there's no actual value supporting this increase since a certain `_amount` of PartyX token is being burnt and the staked balance of the user sacrificing his PartyX tokens increases by 20% `_amount`. The problem arises when users are trying to unstake since the actual balance of PartyX tokens in the pool will be smaller than `totalPartyXStaked`. It will be first come first served and it should not be an intended behavior.

Here's an overview of the sacrifice functionality:

```

function sacrifice(uint256 _amount)

    external

    nonReentrant

    whenNotPaused

{

    require(sacrificeEnabled, "Sacrificing disabled");

    require(_amount > 0, "Invalid amount");

        require(IERC20(address(partyXToken)).balanceOf(msg.sender) >= _amount,
"Insufficient PARTYX balance");


    _safeTransferFrom(address(partyXToken), msg.sender, address(this), _amount);

    IPartyXToken.burn(_amount);

    _mint(msg.sender, ((_amount * sacrificeRate) / 100));

    stakedPartyXBalance[msg.sender] += ((_amount * sacrificeRate) / 100);

    totalPartyXStaked += ((_amount * sacrificeRate) / 100);


    emit Sacrificed(msg.sender, _amount);

}

```

Proof of Concept

Detailed scenario highlighting the vulnerability:

1. Sacrifice rate = 200
2. User A has 15e18 PartyX tokens initially
3. User A stakes 10e18 PartyX tokens
4. User A sacrifices 5e18 PartyX tokens
5. User A now has 10e18 SPLASH tokens and 20e18 staked PartyX
6. The pool has 10e18 PartyX tokens
7. User B stakes 10e18 PartyX tokens

8. The pool now has 20e18 PartyX tokens but has a total staked balance of 30e18 PartyX tokens
9. User A unstakes and claims his unstaked balance successfully
10. User B unstakes successfully but his call to `claimUnstaked()` reverts because this line would fail because of insufficient funds in the `PoolPartyStaking` contract

```
_safeTransfer(address(partyXToken), msg.sender, _amount);
```

Impact

Users who sacrifice will have a greater staked PartyX balance than they should, when unstaking, they will get larger rewards preventing other users from getting their staking rewards. DoS for some users who lose funds.

Recommendation

Remove these 2 lines:

```
stakedPartyXBalance[msg.sender] += ((amount * sacrificeRate) / 100);
totalPartyXStaked += ((amount * sacrificeRate) / 100);
```

Status

Resolved

[H-02] ResellerCommunityRewards#distribute - Users that are not processed in the first batch of distribution will get rewards that are smaller than expected

Description

If the distribution process requires several batches (calls to `distribute()`) to be completed, then the users processed in the 2nd batch and above will receive rewards that are smaller than what they expect and actually deserve and what they could have received had they been processed in the 1st batch.

Vulnerability Details

Here is a simplified version of the `distribute()` function with only the parts that are interesting to explain the vulnerability:



```

function distribute() external nonReentrant onlyOwner returns (uint256) {

    uint256 totalRewards = address(this).balance;

    require(totalRewards > 0, "No rewards to distribute");

    //...

    shareAmount = (totalRewards * ShareHoldersAmounts[shareholder]) / tTotal;

    }

    //...

    (bool success, ) = payable(poolParty).call{value: batchTotal}(""); // @audit-issue
    this will reduce the balance of the contract and in the next call to distribute()
    shareAmount will be smaller

    require(success, "ETH transfer failed");

    // Use the more efficient batchDeposit function

    PoolParty.batchDeposit(recipients, amounts, address(0), address(0));

}

```

1. `totalRewards` is set to `address(this).balance` with every new call to `distribute()`
2. `shareAmount` is proportional to `totalRewards`:
The bigger `totalRewards`, the bigger `shareAmount` and inversely, the smaller `totalRewards`, the smaller `shareAmount`
3. The sum of the `shareAmount` of all processed users (`batchTotal`) is transferred to the `PoolParty` contract reducing `address(this).balance` and thus `totalRewards` in the next calls to `distribute()`
4. `batchDeposit()` is called in order to give the users their rewards

Proof Of Concept

Detailed scenario highlighting the vulnerability:

1. Account with the `DEFAULT_ADMIN_ROLE` calls `PoolPartyStaking::distributeRewards()`



2. This call will trigger `ResellerCommunityRewards::distribute()` in the case where `ResellerCommunityRewards::distribute()` is triggered more than once, the rewards assigned to the users processed in the next calls to `ResellerCommunityRewards::distribute()` will be smaller than expected.

Impact

Users from 2nd batch and above will end up getting smaller rewards than what they actually expect and deserve. They will get smaller rewards compared to the users processed in the 1st batch.

Recommendation

Only execute this line when it's the 1st batch (1st call to `distribute()`)

```
uint256 totalRewards = address(this).balance;
```

Status

Resolved

Medium

[M-01] PoolPartyStaking - Users are not incentivized to stake as staking PartyX tokens offers no rewards

Description

The staking mechanism is wrong, it does not incentivize people to stake PartyX tokens

Vulnerability Details

The issue lies in how the staking mechanism is designed. Stakers are basically locking their PartyX tokens, getting SPLASH tokens that are then burnt when unlocking their stakes. Stakers are not getting rewarded.

Impact

Users might not want to stake their PartyX tokens.

Recommendation

Reward the stakers based on the amount they staked. Redesign the staking mechanism. If this is a design choice and PartyX tokens' only utility is to be converted to SPLASH tokens that would in return be staked. Then it is recommended to change the term staking and start using locking as it is a more adequate description for the PartyX tokens. If this is a design choice I don't how interesting it would be for users to lock and unlock tokens while spending gas in order to get another token that when staked would generate them rewards.

Status

Acknowledged

[M-02] PoolPartySwap - The pausing mechanism is non-functional as `_pause()` and `_unpause()` can never be called as `claimETH()` can be called anytime

Description

The pausing mechanism is non-functional as `claimETH()` can be called anytime.



Vulnerability Details

`_pause()` and `_unpause()` are internal functions and cannot be called directly, the contract will never be paused and `claimETH()` will be callable at anytime since the modifier `whenNotPaused` will never revert.

Impact

Pausing mechanism is not working properly in the `PoolPartySwap` contract as ETH can be claimed anytime through `claimETH()`

Recommendation

Add external functions `pause()` and `unpause()` that would call the internal functions `_pause()` and `_unpause()` respectively.

Status

Resolved

[M-03] ResellerCommunityRewardsL2#convertNativeToStablecoin - No slippage protection on swaps

Description

Lack of slippage protection on swaps might cause the community sellers to transfer more ETH than they need in order to swap them against stablecoins on L2 and bridge them to mainnet.

Vulnerability Details

Here's an overview of the vulnerable function:

```
function convertNativeToStablecoin(uint256 ethAmount) internal returns (uint256
stablecoinAmount) {

    //...

    uniswapRouter.swapExactETHForTokens{ value: ethAmount }(

        0, // Accept any amount (the threshold check is done above)
```

```

        path,

        address(this),

        block.timestamp

    );

    //...

}

```

There is no slippage protection on swaps. Threshold check done above means nothing there can still be disadvantageous swaps for users with no slippage protection even though there was a threshold check done before:

```

function processBridging() internal {

    // Check the current stablecoin balance.

    uint256 currentStables = IERC20(stablecoinAddress).balanceOf(address(this));

    // Set the additional gas for the destination composed call.

    uint256 additionalGas = 3000000; // Adjust as needed.

    if (currentStables >= threshold) {

        // Estimate fee for bridging the entire stablecoin balance.

        uint256 fee = estimateFee(currentStables, additionalGas);

        // Only proceed if there is enough ETH to cover the fee.

        if (address(this).balance >= fee) {

            rideTaxi(currentStables, additionalGas);

        }

        // Convert any remaining ETH to stablecoin

        if (address(this).balance > 0) {

```

```

        convertNativeToStablecoin(address(this).balance);

    }

    return;

} else {

    // Stablecoin balance is below the threshold; swap all available ETH for
stablecoins.

    uint256 availableETH = address(this).balance;

    if (availableETH > 0) {

        convertNativeToStablecoin(availableETH);

    }

    // Do not attempt bridging if the threshold isn't reached.

    return;

}

}

```

Impact

Loss of funds for the reseller community because of a lack of slippage protection on swaps, as bridging a certain amount of stablecoins might require more ETH than expected

Recommendation

Add a rational minAmount instead of 0

```

uniswapRouter.swapExactETHForTokens( value: ethAmount )(

    0, // Fix this line

    path,

    address(this),

    block.timestamp

);

```

Status

Resolved

[M-04] PriceOracleAddresses#getWETHAddress - PUSH0 opcode not supported on FANTOM

Description

PUSH0 opcode is not supported on FANTOM as of mid-2024 so anything related to FANTOM must be removed since we're using solidity version 0.8.29 that might emit the PUSH0 opcode

Vulnerability Details

This function suggests that protocol might be deployed to FANTOM, something that is not advised.

```
function getWETHAddress() internal view returns (address) {  
  
    uint256 chainId;  
  
    assembly {  
  
        chainId := chainid()  
  
    }  
  
    // ...  
  
    } else if (chainId == 250) { // Fantom  
  
        return 0x21be370D5312f44cB42ce377BC9b8a0cEF1A4C83; // WFTM  
  
    }  
  
    //...  
  
}
```

Impact

Deployment failures or runtime failures

Recommendation

Anything related to FANTOM must be removed.

Checkout this website to know which chains support the PUSH0 opcode:
<https://push0.info/>

Status

Resolved

[M-05]RouterAddresses#getRouterAddress,StargateAddresses#getStargatePoolAddresses,StargateAddresses#getStargateRouterAddress,StargateAddresses#getStargateEndpointId,StargateAddresses#getWETHAddress - Some chains have a wrong/outdated chainId

Description

Some chains have wrong and outdated chain IDs, this needs to be fixed to avoid involving chains that were not intended to be involved and avoid scenarios where the chainId is outdated, disabling token addresses fetching. Protocol team most certainly has a list of chains they wish to deploy to and retrieve tokens addresses from.

Impact

Disability of retrieving addresses from some chains because of outdated chain IDs. That's the case of Sei for example.

Recommendation

Fix the chain IDs that need to be fixed as well as the tokens' addresses on each of the chains.

Here are some of the fixes you can already apply (list is not exhaustive):

- Replace 9999 with 81457 the correct chain ID for Blast
- Replace 1261120 with 1329 the correct chain ID for Sei (1261120 is outdated)

- Replace 4337 with 146 the correct chain ID for Sonic (4337 is for beam)

Status

Resolved

[M-06] PoolPartySwap#updateOwner - function is not working as expected as owners might not be able to allow cancellation when it was prevented

Description

Updating ownership does update the owner; however, one thing that it is supposed to do that it doesn't in one particular case is changing cancellation prevention. In fact, it fails to do so when allowing cancellation when it was prevented.

Vulnerability Details

Here's an overview of the issue:

```
function updatePoolOwner(  
    address _token,  
    address _pool,  
    address newOwner,  
    bool preventCancellation  
) external {  
    require(isPool[_pool]);  
    uint256 _amount = IPoolPartyContract(_pool).updateOwner(msg.sender, newOwner,  
false);  
    //...  
    if(preventCancellation) {  
        IPoolPartyContract(_pool).preventCancellation(preventCancellation);  
    }  
    //...
```

```
}
```

```
function updateOwner(
    address owner,
    address newOwner,
    bool _preventCancel
) external onlyRole(POOLPARTY_ADMIN) nonReentrant LockThePool returns(
    uint256 amount
) {
    require(address(owner) == address(poolOwner), "Must be pool owner to update ownership");

    poolOwner = newOwner;

    if (!preventCancel) {
        preventCancel = _preventCancel;
    }

    return(_balance);
}
```

These 2 subcalls are the most crucial to explain the vulnerability:

- `preventCancel` is initially set to true (it will not enter the if scope)
- `preventCancellation` is false (it will not enter the if scope)
- Result: `preventCancel` will be left unchanged

Proof of Concept

Detailed scenario of the vulnerability:

1. Owner A owns pool P where cancellation is prevented
2. Owner A updated pool ownership to Owner B who wishes to allow cancellation



3. Ownership will be successfully updated however cancellation will still be prevented

Impact

The pool owner will not be able to update cancellation prevention for his pool.

Recommendation

Fix the issue to allow ownership updates while successfully deciding whether the pool prevents or allows cancellations. My recommendation would be to remove anything related to cancellation prevention in `ownedPool::updateOwner(...)` and always call `ownedPool::PreventCancellation(...)`

Status

Acknowledged

Low

[L-01] PoolPartyStaking#getTotalStakedAndSacrificed - View function will return wrong values if any user has sacrificed PartyX tokens

Description

The `getTotalStakedAndSacrificed()` returns, as its name suggests, the sum of the total staked and sacrificed balances of PartyX tokens.

Here's an overview of the function:

```
function getTotalStakedAndSacrificed()  
  
    external  
  
    view  
  
    returns (uint256)  
  
    {  
  
        return totalPartyXStaked + totalPartyXSacrificed;  
  
    }
```

Recommendation

The fix is in the `sacrifice(...)` function.

Add the following line:

```
totalPartyXSacrificed += _amount;
```

Don't forget to apply the changes recommended in H-01.

Status

Resolved

[L-02] PoolPartySwap#__PoolParty_init_unchained - Double use of initializer modifier allows reinitialization

Description

The `initializer` modifier is already used for the `initialize(...)` function and this is enough, it shouldn't be used again for the internal function called by `initialize(...)`: `__PoolParty_init_unchained`.

The issue is that the 2nd `initializer` would allow anyone to call `initialize(...)` again while the purpose of `initializer` is to prevent that from happening. This is due to the inner mechanisms of the `initializer` modifier, how it works under the hood.

Recommendation

Remove the `initializer` modifier from the internal function `__PoolParty_init_unchained`.

Status

Resolved

[L-03] PriceOracleAddresses#getWETHAddress - Should revert on unknown chain ID instead of defaulting to address on mainnet

Description

If mainnet chain ID or the other supported chain IDs don't match the `chainID` then it's better to revert. There should be no problem because the contracts that are using the library are deployed on a supported chain but it's still better to revert.

Impact

Very unlikely to occur but if the chain ID does not correspond to any of the supported chain IDs then it's best to revert to prevent unintended events from happening.

Recommendation

Revert in the last `else` block.

Status

Resolved



Gas

[G-01] ResellerCommunityRewards#distribute - LockThePool modifier is useless and can be omitted as lockPool is never checked

Description

The modifier does not provide an extra layer of security against reentrancy and doesn't prevent concurrent executions of modified functions, lockPool is just set to true at the start of the function and to false at the end, it is never checked and is useless the way it is implemented right now

Recommendation

Remove the modifier completely or reimplement it in a useful way if you see fit.

Status

Resolved

[G-02] ResellerCommunityRewardsL2#Constructor - No need for redundant check

Description

This check is redundant and should be removed for better optimization

```
require(stablecoinAddress != address(0), "No supported stablecoin available");
```

In fact, this condition is checked in a subcall from the constructor:
SupportedAssetsManager::getPreferredStablecoin(...)

Recommendation

Remove the following line:

```
require(stablecoinAddress != address(0), "No supported stablecoin available");
```

Status

Resolved

[G-03] PublicPool#checkFeeData - Unintentionally duplicated lines will cost much more gas since they involve writing to the storage

Description

2 lines are unintentionally duplicated inside the `checkFeeData()` function.

Recommendation

Remove the duplicated lines:

```
ResellerFeeReceiver = _ResellerFeeReceiver;
ResellerFee = _ResellerFee;
```

Status

Resolved

[G-04] PublicPool#takeFee - OwnedPool#takeFee - Function needs serious refactoring to be more optimized

Description

`takeFee()` needs serious refactoring in both `PublicPool` and `OwnedPool` contracts, there are too many checks and lines of code, making it expensive in gas and hard to read. Code size can be reduced.

Recommendation

Replace the lines 457->493 of the `PublicPool::takeFee()` function with the following:

```
if(isVerified) {
    (success, ) = payable(ProjectFeeReceiver).call{value: projectFee}("");
    require(success, "Failed to send Project Fee");
}
```

```

    }

    if(isReseller) {

        (success, ) = payable(ResellerFeeReceiver).call{value: resellerFee}("");

        require(success, "Failed to send Reseller Fee");

        (success, ) = payable(ResellerCommunityRewards).call{value: resellerCommunityFee}("");

        require(success, "Failed to send Reseller Fee");

    }

    (success, ) = payable(PoolPartyFeeReceiver).call{value: partyFeeAmount}("");

    require(success, "Failed to send Party Fee");

    return newPrice;

```

It is pretty much the same for the OwnedPool. Do not forget these lines:

```

(success, ) = payable(_PoolPartyInterface).call{value: newPrice}("");

require(success, "Failed to send ETH");

```

Status

Resolved

[G-05] ResellerCommunityRewardsL2#getETHForStablecoin - Function is never called and can be removed

Description

This function is never called and can be removed to reduce the code size and save gas on deployment:

```

function getETHForStablecoin(uint256 desiredStablecoin) internal view returns (uint256
ethNeeded) { // @audit-info never called

    address[] memory path = new address[](2);

    path[0] = SupportedAssetsManager.getWrappedNative(chainId);

```

```

    path[1] = stablecoinAddress;

    uint256[] memory amounts = uniswapRouter.getAmountsIn(desiredStablecoin, path);

    ethNeeded = amounts[0];

}

```

Recommendation

Remove the function completely.

Status

Resolved

[G-06] PoolPartyStaking#toHexString - Function can be more optimized

Description

This function can be way more optimized:

```

function toHexString(address addr) internal pure returns (string memory) {
    //@audit-info can be much more optimized, powers are too expensive

    bytes memory buffer = new bytes(42);

    buffer[0] = '0';

    buffer[1] = 'x';

    bytes memory alphabet = "0123456789abcdef";

    for (uint256 i = 0; i < 20; i++) {

        buffer[2+i*2] = alphabet[uint8(uint256(uint160(addr)) / (16**(39-i*2)) %
16)];

        buffer[3+i*2] = alphabet[uint8(uint256(uint160(addr)) / (16**(38-i*2)) %
16)];

    }

    return string(buffer);

}

```

Recommendation

Make the following changes:

```
function toHexString(address addr) internal pure returns (string memory) {

    bytes memory alphabet = "0123456789abcdef";

    bytes20 value = bytes20(addr);

    bytes memory str = new bytes(42);

    str[0] = '0';

    str[1] = 'x';

    for (uint256 i = 0; i < 20; i++) {

        str[2 + i * 2] = alphabet[uint8(value[i] >> 4)];

        str[3 + i * 2] = alphabet[uint8(value[i] & 0x0f)];

    }

    return string(str);
}
```

Status

Resolved

[G-07] PublicPool#cancelPool - Unnecessary check

Description

This function contains an unnecessary check(if the 1st require succeeded than the 2nd one will as well):

```
function cancelPool(

    address User

) external onlyRole(P00LPARTY_ADMIN) returns(

    uint256 amount,

    bool hidePool

) {
```

```

        require(poolContributor[User] && contributorToIndex[User] != 0, "Only pool
        contributors can call this function");

        amount = contributorsList[contributorToIndex[User]].amount;

        require(amount > 0, "No tokens to withdraw");

        //...
    }

```

Recommendation

Remove the unnecessary check:

```

require(amount > 0, "No tokens to withdraw");

```

Status

Resolved

[G-08] [StargateAddresses#getStargatePoolAddresses,SupportedAssetsManager#getUSDTAddress,SupportedAssetsManager#getUSDCAddress](#) - Unnecessary multiple long functions for a functionality that needs 1 function

Description

The way the project is checking the availability of stablecoins on each chain can be much more optimized, 3 functions for this mechanism is way too much.

Recommendation

Make the process smaller.

Status

Resolved

[G-09] ownedPool - Unused state variable can be removed to reduce the storage

Description

The variable `totalTwitterReserved` variable is never used and can be removed to reduce the storage.

Recommendation

Either you can make `totalTwitterReserved` public and update it in the `setTwitterUsers()` function or remove it completely

Status

Resolved

[G-10] publicPool#poolTransfer - Unused state variable can be removed to reduce the storage

Description

The variable `isComplete` variable is only set to `false` once and never checked, it can be removed to reduce the storage.

Recommendation

Remove it completely or set it to `true` once the transfer is complete.

Status

Resolved

QA

[Q-01] Fix the naming conventions through the whole project

Description

Some contracts' names start with a lower case letter, some external functions start with an underscore (`\_`), some variables start with an upper case letter.

Recommendation

Fix the naming conventions for better readability, here's an unexhaustive list of advice:

- Internal and private functions should start with an underscore
- Contracts' names should be in Pascal case
- Variable's names should be in lower-camel case
- Make sure interfaces have *interface* in their names, addresses on the contrary should not have *interface* in their names
- Avoid having your error messages named *error* in try catches since it's a keyword
- Make sure people can tell what the function does from its name, if a function's name is: *stake()* it should stake and generate a reward, not just lock tokens, if the intended behavior is to lock tokens then the function should be named: *lock()*
- Fix the natspec of some functions like `setFixedPrice()`

Status

Acknowledged

[Q-02] PoolPartySwap - Make sure `_disableInitializers()` is used for upgradeable contracts

Description

`_disableInitializers()` is used to lock the implementation contract of an upgradeable smart contract, preventing it from being initialized directly. This is a security best practice in the proxy pattern.

Recommendation

Use `_disableInitializers()` for upgradeable contracts, you already do it in `PoolPartyStaking`, do it as well in `PoolPartySwap`

Status

Resolved

[Q-03] UpdatePoolPartyStruct#_updateReservedAmount - Make sure to never remove from an empty list and remove from a non-empty list instead

Description

Check out this issue: (there are 2 instances)

```
if(Handle.offeredTokens.length() == 0) { // ok @audit-issue can't remove from an empty
list
    Handle.offeredTokens.remove(token);
}
```

Recommendation

Check if list is non-empty and remove

Status

Resolved

[Q-04] Fix event emissions through the contract

Description

Put the exact fee instead of 0.

```
emit SwapAndBridge(ethAmount, stablecoinAmount, 0);
```

Only emit this event when cancellation prevention happens:

```
emit PoolPartyEvents.preventCancellation(_pool, msg.sender, preventCancellation);
```



Recommendation

Emit the events when you should and with the correct parameters

Status

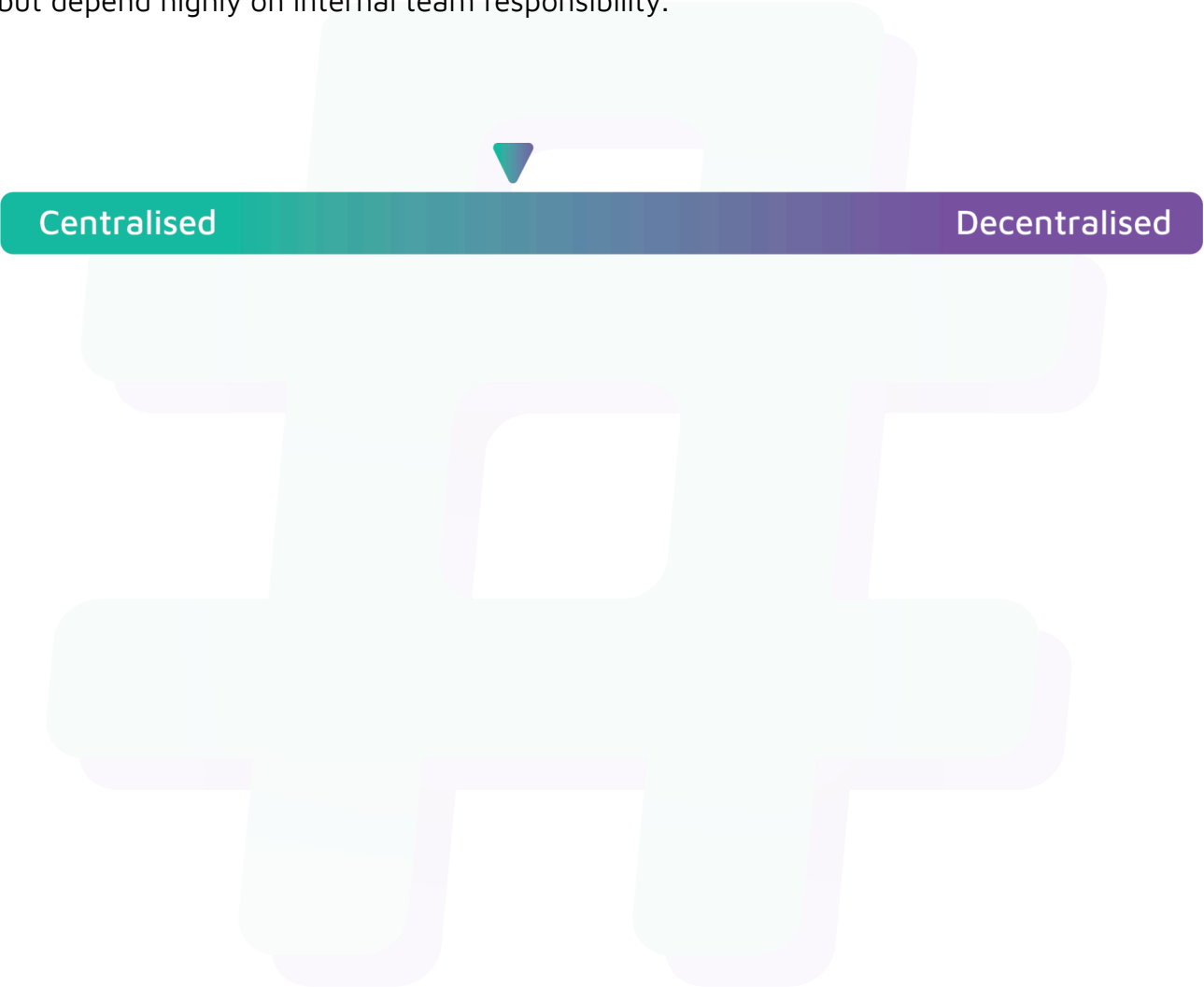
Acknowledged



Centralisation

The PoolParty project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

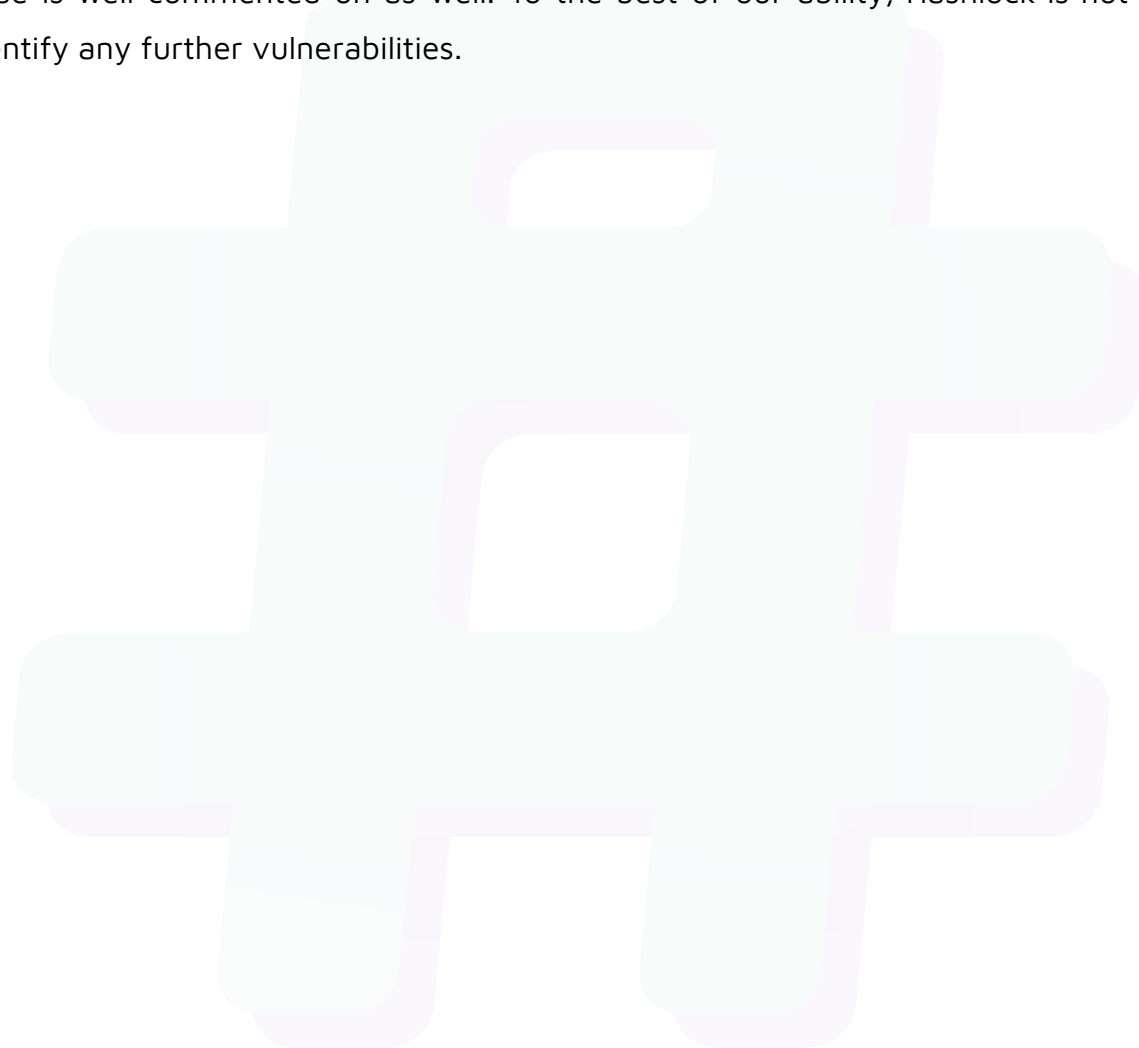


Centralised

Decentralised

Conclusion

After Hashlock's analysis, the PoolParty project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.



Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.